Formulación y Búsqueda Ciega en Espacios de Estados: el Problema de los Misioneros y Caníbales con BFS y DFS

Lenin Alexander Amangandi LLumiguano

Escuela Politécnica Nacional, Facultad de Ingeniería de Sistemas,
Inteligencia Artificial (ICCD523), Quito, Ecuador
lenin.amangandi@epn.edu.ec
https://github.com/Lenin27Amangandi/IA_2k26_Individual

Resumen En este informe se formaliza el problema clásico de los misioneros y caníbales como un problema de búsqueda en un espacio de estados, se implementa en Python sin interfaz gráfica y se comparan empíricamente las estrategias de búsqueda no informada BFS (cola) y DFS (pila). Los estados se modelan como tuplas (m, c, b) y se define de forma explícita el estado inicial, la prueba de meta, el conjunto de acciones y el modelo de transición. Se mide el número de nodos expandidos, el tiempo de ejecución, la longitud del camino y, como medidas de memoria, el tamaño máximo de la frontera y el pico de memoria reservado por Python. En esta instancia, BFS expandió 14 nodos y DFS 11, y ambos hallaron una solución de 11 acciones. Se discute por qué este resultado no contradice los resultados asintóticos conocidos y qué garantías ofrece cada algoritmo.

Keywords: Búsqueda no informada · Espacio de estados · BFS · DFS · Misioneros y caníbales.

1. Introducción y Marco Teórico

El problema de los misioneros y caníbales es un problema clásico de inteligencia artificial: tres misioneros y tres caníbales deben cruzar un río en un bote con capacidad para dos personas, y en ninguna orilla los caníbales pueden superar en número a los misioneros cuando hay al menos un misionero presente. Amarel [2] lo utilizó para estudiar cómo la elección de la representación de un problema afecta la facilidad para resolverlo, y Russell y Norvig [1] lo emplean como ejemplo de formulación de problemas de búsqueda.

Según Russell y Norvig [1], una estrategia de búsqueda no informada (o ciega) no dispone de información sobre qué tan cerca está un estado de la meta; solo distingue entre estados y puede generar sucesores. En este trabajo se comparan dos de ellas:

1. **Búsqueda en anchura (BFS):** expande primero los nodos menos profundos mediante una cola FIFO. Es completa y, cuando todas las acciones tienen el mismo costo, encuentra una solución de menor número de pasos [1].

2. **Búsqueda en profundidad (DFS):** expande el nodo más profundo de la frontera mediante una pila LIFO. No garantiza encontrar la solución más corta [1].

Ambos recorridos sobre un grafo explícito con $|V|$ vértices y $|E|$ aristas se ejecutan en tiempo $O(|V| + |E|)$ cuando se registran los vértices ya visitados [3].

2. Formulación Formal del Problema

Sea S el espacio de estados. Cada estado es una tupla

$$s = (m, c, b), \qquad m, c \in \{0, 1, 2, 3\},\ b \in \{0, 1\}, \tag{1}$$

donde m y c son el número de misioneros y de caníbales en la orilla izquierda, y $b = 1$ indica que el bote está en la orilla izquierda ($b = 0$ en la derecha). En la orilla derecha hay $3 - m$ misioneros y $3 - c$ caníbales.

Restricción de seguridad. Un par (m, c) es seguro si ningún misionero puede ser comido en ninguna orilla:

$$\text{Seguro}(m, c) \iff \big(m = 0 \ \vee \ m \geq c\big) \ \wedge \ \big(m = 3 \ \vee \ c \geq m\big). \tag{2}$$

El espacio de estados admisible es $S = \{(m, c, b) \mid \text{Seguro}(m, c)\}$. Por enumeración se obtienen 10 pares (m, c) seguros, es decir, $|S| = 20$ estados válidos, de los cuales 16 son alcanzables desde el estado inicial.

Estado inicial y estado objetivo.

$$s_0 = (3, 3, 1), \qquad s_{\text{meta}} = (0, 0, 0). \tag{3}$$

Conjunto de acciones. Una acción $a = (a_m, a_c)$ indica cuántos misioneros y cuántos caníbales cruzan en el bote. Como el bote lleva como máximo dos personas y no puede cruzar vacío:

$$A = \{(a_m, a_c) \in \mathbb{N}^2 \mid 1 \leq a_m + a_c \leq 2\} = \{(1, 0), (0, 1), (1, 1), (2, 0), (0, 2)\}. \tag{4}$$

Modelo de transición. Sea $\sigma(b) = +1$ si $b = 1$ y $\sigma(b) = -1$ si $b = 0$. La función de resultado es

$$\text{Result}\big((m, c, b), (a_m, a_c)\big) = \big(m - \sigma(b)\, a_m,\ c - \sigma(b)\, a_c,\ 1 - b\big), \tag{5}$$

y la acción es aplicable en $s = (m, c, b)$ solo si el estado resultante (m', c', b') cumple $0 \leq m' \leq 3$, $0 \leq c' \leq 3$ y $\text{Seguro}(m', c')$. La primera condición equivale a exigir que haya suficientes personas en la orilla donde está el bote.

Prueba de meta y costo de camino. El estado s es meta si y solo si $s = (0, 0, 0)$. Cada cruce tiene costo unitario, $c(s, a, s') = 1$, por lo que el costo de una solución es su número de acciones.

3. Implementación

El script `misioneros_canibales.py` no usa interfaz gráfica y modela los estados como tuplas `(misioneros_izq, canibales_izq, bote_izq)`. Las decisiones de diseño relevantes son:

- Cada nodo de la frontera es la pareja (estado, camino), donde el camino es la lista de pares (acción, estado) recorridos.
- **BFS** usa `collections.deque` como cola y marca un estado como visitado al *generarlo*.
- **DFS** usa una lista como pila y marca un estado como visitado al *extraerlo*.
- Un nodo *expandido* es aquel al que se le generan sucesores; el nodo meta no se cuenta como expandido.
- Las acciones se generan en el orden $[(1,0),(0,1),(1,1),(2,0),(0,2)]$. Como el orden afecta a DFS, se incluye además una ejecución con el orden inverso.
- El tiempo se promedia sobre 2000 ejecuciones con `time.perf_counter`, porque una ejecución dura microsegundos. El pico de memoria se mide con `tracemalloc` en una ejecución aparte.
- Cada solución se verifica paso a paso con `verificar_solucion`.

Listing 1.1. Núcleo de BFS (cola) y DFS (pila).

```
def bfs(acciones=ACCIONES):
    frontera = deque()
    frontera.append((ESTADO_INICIAL, []))
    visitados = set()
    visitados.add(ESTADO_INICIAL)
    expandidos = 0
    while len(frontera) > 0:
        estado, camino = frontera.popleft()
        if estado == ESTADO_META:
            return camino, expandidos
        expandidos += 1
        for accion, nuevo in obtener_sucesores(estado, acciones):
            if nuevo not in visitados:
                visitados.add(nuevo)
                frontera.append((nuevo, camino + [(accion, nuevo)]))

def dfs(acciones=ACCIONES):
    frontera = [(ESTADO_INICIAL, [])]
    visitados = set()
    expandidos = 0
    while len(frontera) > 0:
        estado, camino = frontera.pop()
        if estado in visitados:
            continue
        visitados.add(estado)
        if estado == ESTADO_META:
            return camino, expandidos
```

```
        expandidos += 1
        for accion, nuevo in obtener_sucesores(estado, acciones):
            if nuevo not in visitados:
                frontera.append((nuevo, camino + [(accion, nuevo)]))
```

El listado muestra solo el núcleo de ambos algoritmos (se omite el registro del tamaño máximo de la frontera); el código completo está en el repositorio del autor. Un segundo script, `visualizar_benchmark.py`, lee los resultados guardados en un archivo CSV y los presenta en una tabla ordenada y en gráficos.

4. Benchmarking y Resultados

Los algoritmos se ejecutaron sobre la única instancia del problema (3 misioneros, 3 caníbales, bote de capacidad 2). Los resultados se resumen en la Tabla 1 y en la Figura 1.

Cuadro 1. Comparación empírica de BFS y DFS (tiempo promedio de 2000 ejecuciones).

Algoritmo	Nodos expand.	Tiempo (s)	Camino (acc.)	Frontera máx.	Mem. pico (KB)
BFS	14	$1{,}96 \times 10^{-5}$	11	3	1,78
DFS	11	$1{,}63 \times 10^{-5}$	11	5	1,14
DFS (orden inverso)	13	$1{,}85 \times 10^{-5}$	11	3	1,08

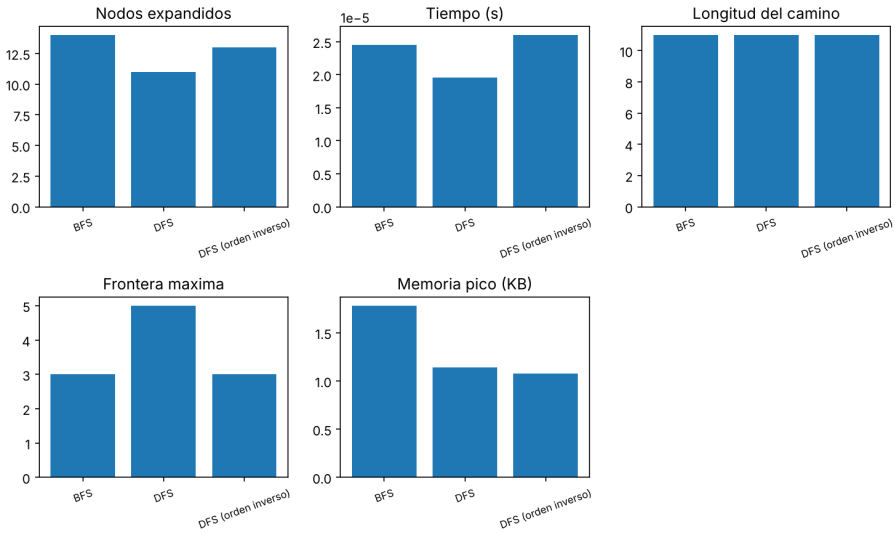


Figura 1. Métricas del benchmark para BFS, DFS y DFS con el orden de acciones invertido.

La solución óptima encontrada por BFS tiene 11 acciones y se muestra en la Tabla 2. Las tres ejecuciones devolvieron soluciones válidas de 11 acciones según `verificar_solucion`.

Cuadro 2. Secuencia de acciones de la solución hallada por BFS (acción = (a_m, a_c)).

Paso	Sentido del bote	Acción	Estado resultante
1	izq. → der.	$(1,1)$	$(2,2,0)$
2	der. → izq.	$(1,0)$	$(3,2,1)$
3	izq. → der.	$(0,2)$	$(3,0,0)$
4	der. → izq.	$(0,1)$	$(3,1,1)$
5	izq. → der.	$(2,0)$	$(1,1,0)$
6	der. → izq.	$(1,1)$	$(2,2,1)$
7	izq. → der.	$(2,0)$	$(0,2,0)$
8	der. → izq.	$(0,1)$	$(0,3,1)$
9	izq. → der.	$(0,2)$	$(0,1,0)$
10	der. → izq.	$(1,0)$	$(1,1,1)$
11	izq. → der.	$(1,1)$	$(0,0,0)$

5. Conclusión Crítica

1. **Teoría frente a la práctica real.** Aunque los libros nos enseñan que BFS y DFS se comportan de manera muy distinta según el crecimiento del pro-

blema, en este ejercicio en particular (que cuenta con apenas 16 estados posibles) esas diferencias teóricas casi no se aprecian. Como el espacio de búsqueda es tan pequeño, ambos algoritmos resolvieron el problema de forma casi instantánea en el orden de microsegundos, con DFS explorando 11 nodos y BFS 14. Las grandes ventajas de memoria y tiempo que predice la teoría solo se vuelven evidentes en problemas gigantes o tableros extensos, pero para un caso de juguete tan acotado, los resultados prácticos son prácticamente iguales.

2. **¿DFS garantizó el camino más corto?** DFS devolvió un camino de 11 acciones, igual al de BFS, y BFS es óptimo en número de pasos porque todas las acciones cuestan lo mismo [1]. Por tanto, en esta instancia DFS *encontró* una solución óptima, pero no la *garantizó*: DFS no ofrece esa garantía en general [1], y el resultado se debe a la estructura de este grafo, no a una propiedad del algoritmo. Obsérvese además que las soluciones difieren en sus acciones: DFS comenzó cruzando dos caníbales, mientras que BFS cruzó un misionero y un caníbal, aunque ambas tienen 11 pasos.
3. **¿BFS consumió más memoria?** Aunque el pico de memoria medido fue ligeramente mayor en BFS (1,78 KB) que en DFS (1,14 KB), la cantidad máxima de nodos en la frontera fue menor en BFS (3 frente a 5). Estas mínimas diferencias de alrededor de 1 KB dependen más de la implementación que del algoritmo. Con un problema tan pequeño no es posible asegurar que BFS consuma más memoria; las ventajas de espacio de DFS descritas en la teoría solo se hacen evidentes en espacios de estados mucho mayores.

Limitaciones. Se evaluó una única instancia, con 16 estados alcanzables; los tiempos provienen de una sola máquina y no se calcularon intervalos de confianza. Una extensión natural es generalizar el número de personas y la capacidad del bote para observar el crecimiento de los nodos expandidos.

Referencias

1. Russell, S., Norvig, P.: Artificial Intelligence: A Modern Approach. 4th edn. Pearson, Hoboken, NJ (2021)
2. Amarel, S.: On representations of problems of reasoning about actions. In: Michie, D. (ed.) Machine Intelligence 3, pp. 131–171. Edinburgh University Press, Edinburgh (1968)
3. Cormen, T.H., Leiserson, C.E., Rivest, R.L., Stein, C.: Introduction to Algorithms. 4th edn. MIT Press, Cambridge, MA (2022)